BINF 4070 / COMS 4995 W008

The Future of Personal Health Assistant

Fall 2026

Time Series and Image Data

Week 07

Dr. Xuhai “Orson” Xu | Dr. Eugene Wu

AI @ VP&S Initiative

COLUMBIA | VAGELOS COLLEGE OF PHYSICIANS AND SURGEONS

COLLABORATORY AT COLUMBIA UNIVERSITY

DAP Lab

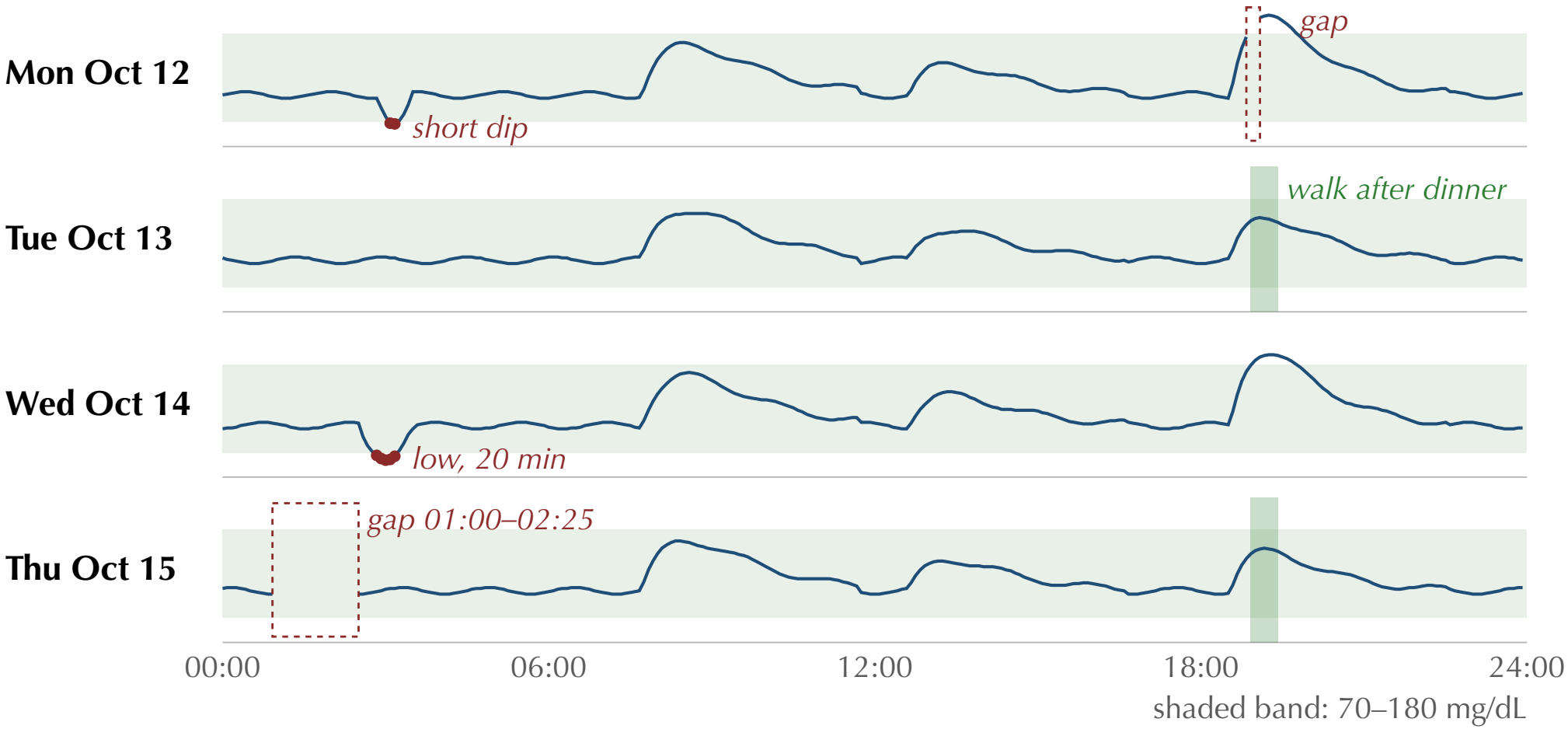
Today's Outline

1. **Time-Series Data**
2. **Time-Series Operations**
3. **Image Data**
4. **Image Operations**

Time-Series Data

Readings ordered by time.

Four Days of Glucose Readings



A continuous glucose monitor (CGM) reports one value every 5 minutes: 288 readings a day, when nothing is missing.

Definition: Time Series

A **time series** is a sequence of observations of one quantity, ordered by time.

Modeled as relation (in practice, stored more efficiently).

person_id	device_id	observed_at	mg_dl
P001	cgm-7731	2026-10-12 18:40:00–04	143
P001	cgm-7731	2026-10-12 18:45:00–04	162
P001	cgm-7731	2026-10-12 18:50:00–04	176

- **Row grain:** one reading from one device
- **Primary key:** (`person_id`, `device_id`, `observed_at`)
- Relations are unordered, but time series are ordered by time.

What People Want from Time Series

Goal	Example question
Summarize	What was my average glucose each day?
Compare	Was this week higher than last week?
Detect events	Did I have any lows overnight?
Relate to other events	Does walking after dinner lower the peak?
Find similar patterns	When else did my glucose look like this?
Forecast	Where will my glucose be in 30 minutes?

Every goal assumes the readings are complete and correctly timed. Often they are not.

Where Time Series Come From

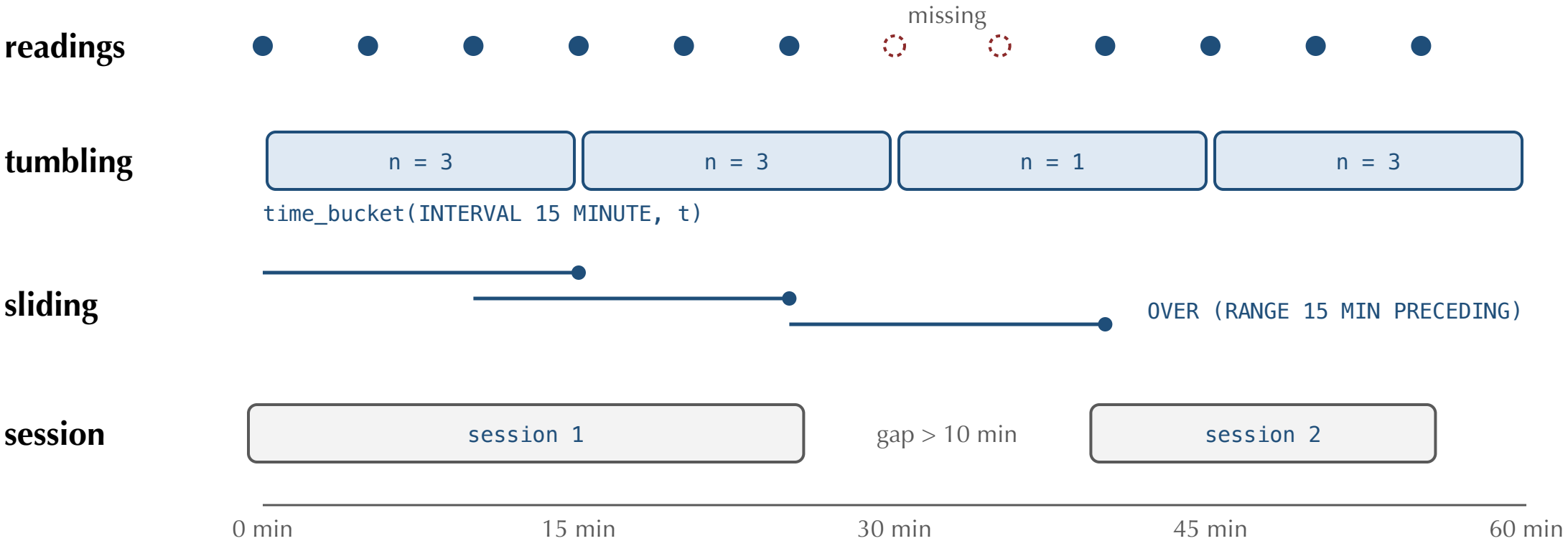
Time series often come from **sensors** (a CGM, a watch) and **logs** (meals, doses), with common errors:

Error	Example	Effect
Gaps	warm-up, lost signal, watch off the wrist	unknown, not 0
Irregular sampling	more heart-rate readings during exercise	unequal counts per window
Duplicates	re-upload after a failed sync	inflated counts and averages
Late arrivals	the phone syncs hours later	yesterday's total changes
Clocks and time zones	no UTC offset; daily buckets in UTC	"Monday" gets 238 of 286 readings
Sensor artifacts	lying on a CGM during sleep	false lows

Time-Series Operations

Windows, gaps, alignment, events, and similarity.

Window Types



A **window** is the set of readings an aggregate summarizes. The window type decides which readings are in it.

Tumbling Windows: time_bucket

```
SELECT time_bucket(INTERVAL 1 HOUR, observed_at) AS hour,
       round(avg(mg_dl), 1) AS mean, min(mg_dl) AS min, max(mg_dl) AS max,
       count(*) AS n                                    -- readings behind each mean
FROM glucose
WHERE observed_at >= '2026-10-12 18:00:00-04' AND observed_at < '2026-10-12 21:00:00-04'
GROUP BY hour ORDER BY hour;
```

hour	mean	min	max	n
18:00	120.3	100	176	11
19:00	191.7	170	203	11
20:00	146.3	134	165	12

Two readings are missing; `n` shows which hours are short.

Sliding Windows: OVER

```
SELECT observed_at, mg_dl,
  avg(mg_dl) OVER (ORDER BY observed_at RANGE BETWEEN INTERVAL 15 MINUTE PRECEDING AND CURRENT ROW),
  avg(mg_dl) OVER (ORDER BY observed_at ROWS BETWEEN 3 PRECEDING AND CURRENT ROW)
FROM glucose WHERE ...;
```

- `avg(…) OVER (…)` computes a value **for every row** from a **frame** of nearby rows; unlike `GROUP BY`, no rows are collapsed
- `RANGE … INTERVAL 15 MINUTE PRECEDING`: the rows in the last 15 minutes of time
- `ROWS … 3 PRECEDING`: the current row and the 3 rows before it, however far back

time	mg/dL	`RANGE` 15 min	`ROWS` 4
18:50	176	149.8	149.8
19:05	200	**188.0**	**170.3**

At 19:05, `RANGE` averages 18:50 and 19:05; `ROWS` reaches back to 18:40, across the gap.

Window Functions: PARTITION BY

```
SELECT person_id, t, mg_dl, row_number() OVER w AS n
FROM readings
WINDOW w AS (PARTITION BY person_id ORDER BY t);
```

- `WINDOW w AS (…)` names a window so several functions can share it: `OVER w`
- `PARTITION BY person_id`: each person's readings are a separate series; a window function never crosses partitions
- `ORDER BY t`: the order within each partition

person_id	t	mg_dl	n
P001	08:00	110	1
P001	08:05	118	2
P002	08:00	95	1
P002	08:05	99	2

Consecutive Readings: LAG

`lag(x) OVER w` is the value of `x` in the **previous row** of the same partition; the first row gets NULL.

```
SELECT observed_at, mg_dl,
       mg_dl - lag(mg_dl) OVER w             AS change,
       observed_at - lag(observed_at) OVER w AS since_prev
FROM glucose
WINDOW w AS (PARTITION BY person_id ORDER BY observed_at);
```

time	mg/dL	change	since_prev
18:45	162	19	00:05:00
18:50	176	14	00:05:00
19:05	200	24	**00:15:00**
19:10	202	2	00:05:00

The change of 24 took 15 minutes, not 5. `WHERE since_prev > INTERVAL 5 MINUTE` lists the gaps.

Aligning Series: ASOF JOIN

An **ASOF JOIN** matches each left row to the **one** right row with the nearest time **at or before** it, among rows that pass the equality conditions.

`meals`

eaten_at	meal
Mon 18:32	dinner

`glucose`

observed_at	mg_dl
Mon 18:25	101
Mon 18:30	100
Mon 18:35	118

result

eaten_at	reading_at	at_meal
Mon 18:32	18:30	100

```
SELECT m.eaten_at, g.observed_at AS reading_at, g.mg_dl AS at_meal
FROM meals AS m ASOF JOIN glucose AS g
  ON m.person_id = g.person_id AND m.eaten_at >= g.observed_at;
```

An equality join on time would return no rows: the two tables never share a timestamp.

Detecting Events

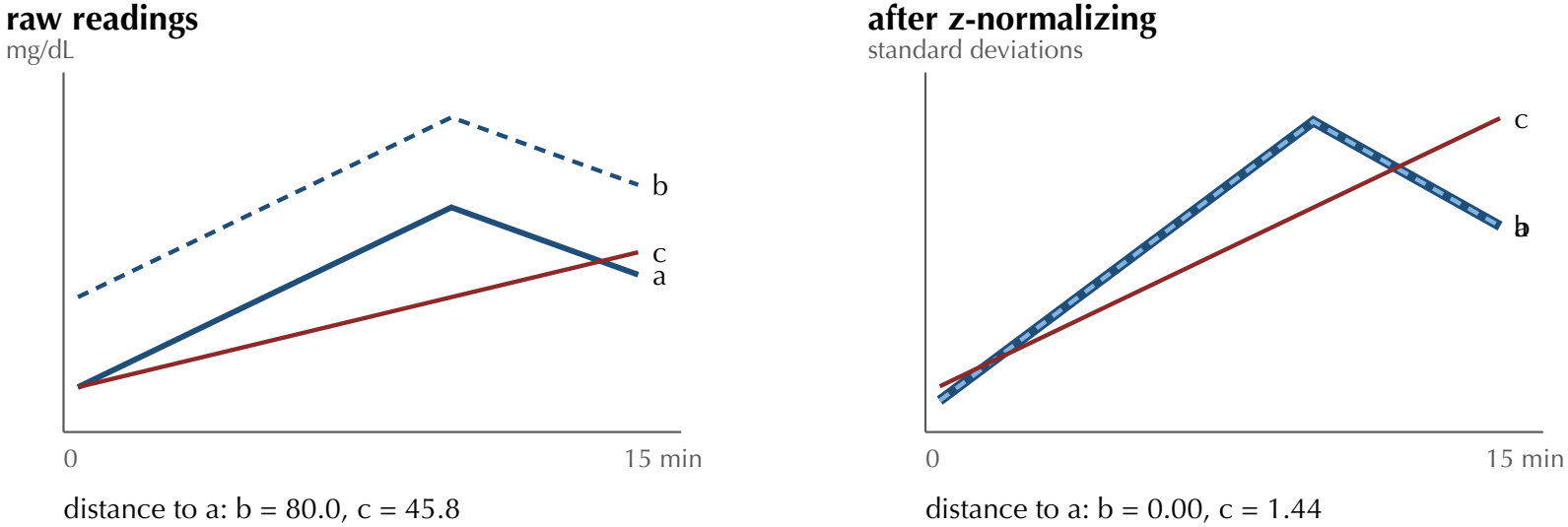
Rule: three consecutive readings below 70 mg/dL (15 minutes) is a low-glucose event.

Wed	02:45	02:50	02:55	03:00	03:05	03:10	03:15
mg/dL	72	**67**	**63**	**61**	**62**	**66**	73

```
SELECT observed_at, mg_dl
FROM (SELECT *, lag(mg_dl, 1) OVER w AS prev1,     -- the previous reading
                lag(mg_dl, 2) OVER w AS prev2      -- the one before that
      FROM glucose WINDOW w AS (PARTITION BY person_id ORDER BY observed_at))
WHERE mg_dl < 70 AND prev1 < 70 AND prev2 < 70;
```

Returns Wed 03:00, 03:05, and 03:10: the readings that complete a run of three lows. Monday's two low readings do not.

Similarity Search and z-Normalization



- **z-normalizing** subtracts each window's mean and divides by its standard deviation, so distance compares shape, not level
- Use it when shape matters at any level: a post-meal curve, an ECG beat, a gait cycle. Skip it when the level matters: time above 180 mg/dL
- Each window is a vector, so the Week 6 vector search applies

Break

5 minutes

Next: images.

Image Data

Scans, photos, and form images.

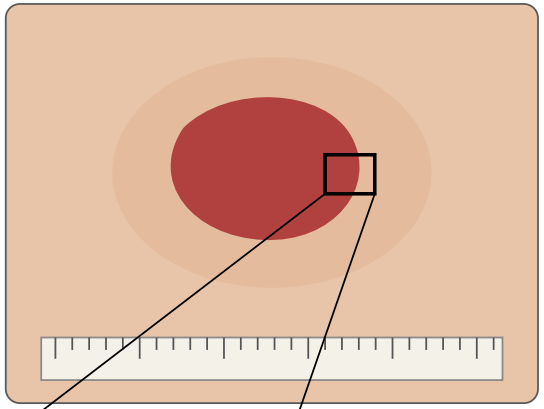
Three Kinds of Health Images

	Scan	Photo	Form image
Example	fundus photograph, X-ray, CT	wound, skin, or meal photo	faxed or scanned lab report
Produced by	a clinical device, under a protocol	a phone, in uncontrolled light	a scanner or a fax machine
Format	DICOM	JPEG or HEIC with EXIF	PDF, TIFF, or JPEG
Useful content	pixel values and acquisition metadata	appearance, with unknown scale and color	the text and its layout
Typical question	has the lesion changed?	is the wound smaller?	what value was reported?

Definition: Image Data

An **image** is a pixel array (rows × columns × channels) plus **metadata** about who, when, and how it was acquired. Labels, regions, text, and embeddings are **derived data**.

pixels



176,64,62	180,66,63	201,150,120
172,61,60	178,65,61	198,146,117
169,60,58	190,98,80	203,152,122

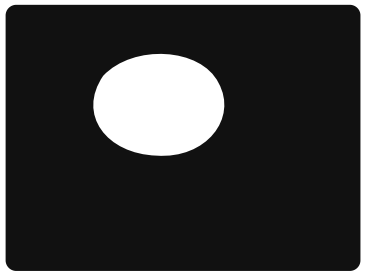
3024 × 4032 pixels, 3 numbers each (red, green, blue)

metadata: images

image_id	I5
person_id	P001
captured_at	2026-09-20 21:14
device	phone
mm_per_pixel	0.2
path	photos/IMG_4417.jpg

derived: segments

seg	label	pixels	area	mask
S1	wound	12,500	5.0 cm²	S1.png
S2	ruler	41,000	–	S2.png



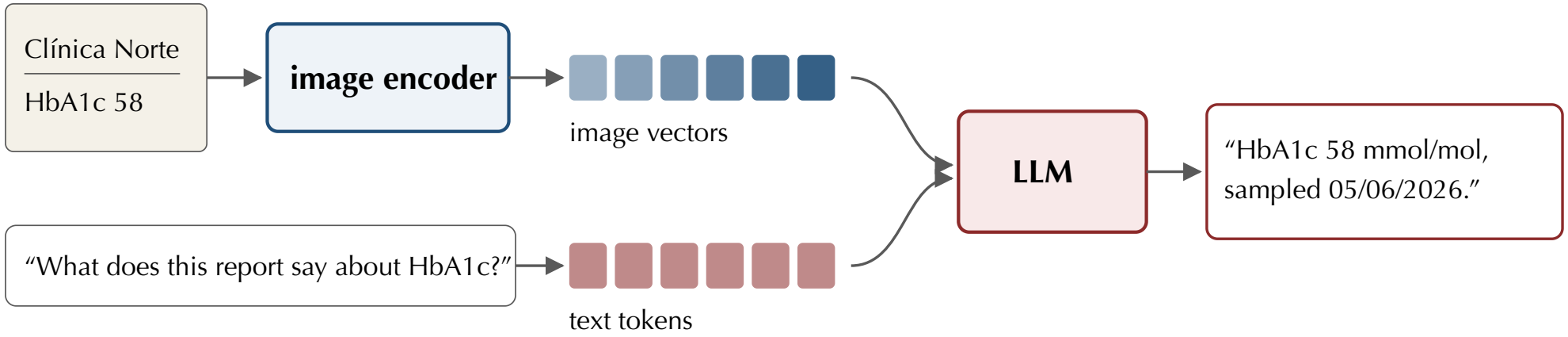
S1.png: the mask, white where the pixel is wound

image_id I5 links every derived row to its image

Image Operations

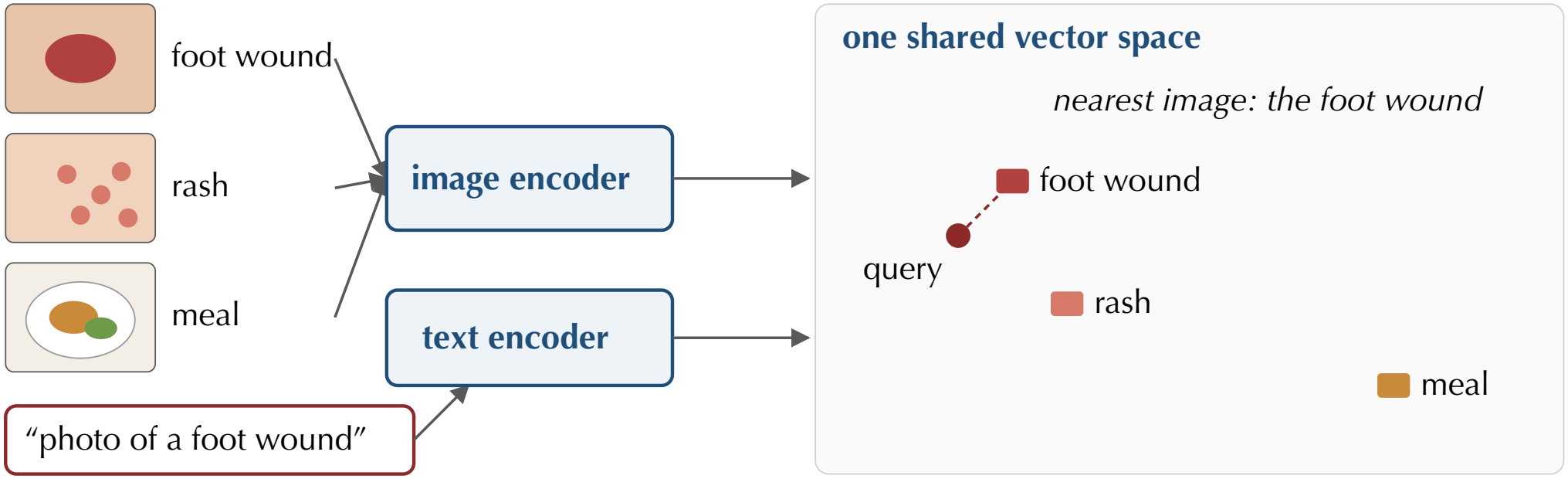
Search, recognition, and extraction.

Vision-Language Models



- A **vision-language model (VLM)** is an LLM that also reads images: an image encoder turns the image into vectors that the LLM reads alongside the text tokens
- It can describe a photo, answer questions about it, or extract fields from a form
- Its output is generated text, so it can be fluent and wrong

Image Embeddings



- Trained on image–caption pairs, an image encoder and a text encoder share one vector space
- A text query finds images, and an image finds similar images: the Week 6 search, unchanged

Image Search with OpenAI

OpenAI's embedding models take text only, so a vision model describes each photo, and the description is embedded:

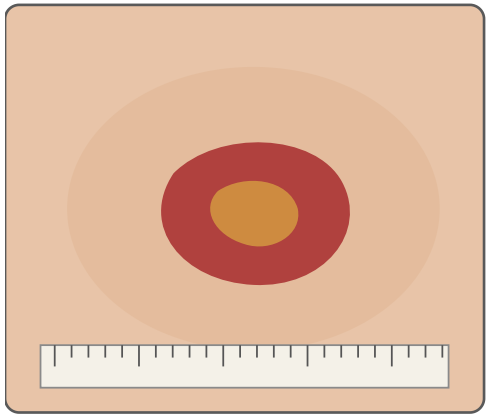
```
def describe(path):                                          # VLM: photo → one-sentence description
    b64 = base64.b64encode(open(path, "rb").read()).decode()
    r = client.chat.completions.create(model="gpt-5.6-luna", messages=[{"role": "user", "content": [
        {"type": "text", "text": "Describe this health photo in one sentence."},
        {"type": "image_url", "image_url": {"url": f"{b64}"}}]}])
    return r.choices[0].message.content

for image_id, path in con.sql("SELECT image_id, path FROM images").fetchall():
    caption = describe(path)
    vec = client.embeddings.create(model="text-embedding-3-small", input=caption).data[0].embedding
    con.execute("UPDATE images SET caption = ?, embedding = ? WHERE image_id = ?", [caption, vec, image_id])
```

```
SELECT image_id, caption, array_cosine_similarity(embedding, $q) AS sim   -- FLOAT[1536]
FROM images ORDER BY sim DESC LIMIT 5;
```

Recognition Tasks

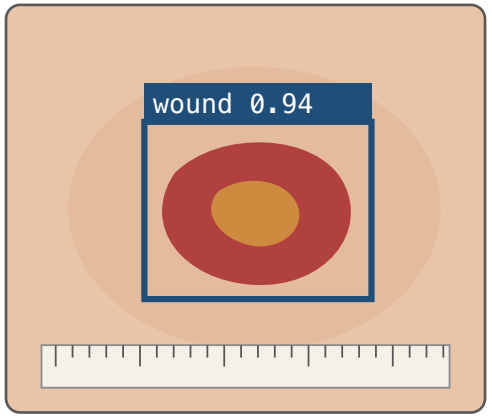
Classification



one label per image

`"ulcer", p = 0.93`

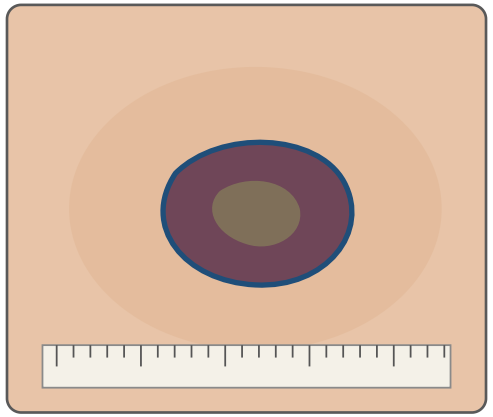
Detection



a box per object

`x, y, w, h, label, score`

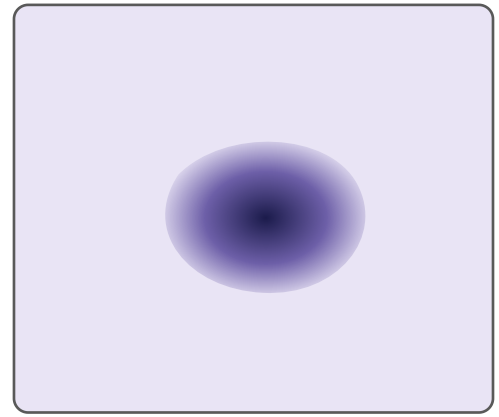
Segmentation



a label per pixel

`mask → area in mm²`

Depth



a distance per pixel

`relative depth map`

Each output is stored as rows with the model version, as with the Week 5 semantic columns.

A Detector in SQL

Register a detection model as a DuckDB function, then call it like any other:

```
from ultralytics import YOLO
model = YOLO("wound_detector.pt")                    # a YOLO model fine-tuned on wound photos

def detect(path):                                    # one dict per detected box
    r = model(path)[0]
    return [{"label": r.names[int(c)], "x": x1, "y": y1, "w": x2 - x1, "h": y2 - y1, "score": s}
            for (x1, y1, x2, y2), c, s in zip(r.boxes.xyxy.tolist(), r.boxes.cls.tolist(), r.boxes.conf.tolist())]

box = duckdb.struct_type({"label": "VARCHAR", "x": "DOUBLE", "y": "DOUBLE",
                          "w": "DOUBLE", "h": "DOUBLE", "score": "DOUBLE"})
con.create_function("detect", detect, ["VARCHAR"], duckdb.list_type(box))
```

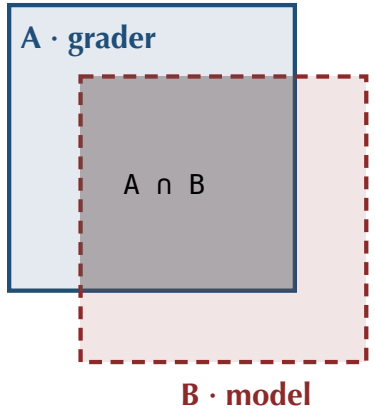
```
CREATE TABLE detections AS
SELECT image_id, unnest(detect(path), recursive := true) FROM images;   -- one row per box
```

Detection: Boxes as Rows

`detections`

image_id	label	x	y	w	h	score
I5	wound	410	380	260	210	0.94
I5	ruler	120	900	820	70	0.88
I7	wound	430	395	220	180	0.91

- **IoU** (intersection over union) scores whether a model's box matches a grader's; ≥ 0.5 usually counts as correct
- **Segmentation** returns a mask instead of a box: a non-rectangular region, stored as a mask image plus a row with its pixel count and area

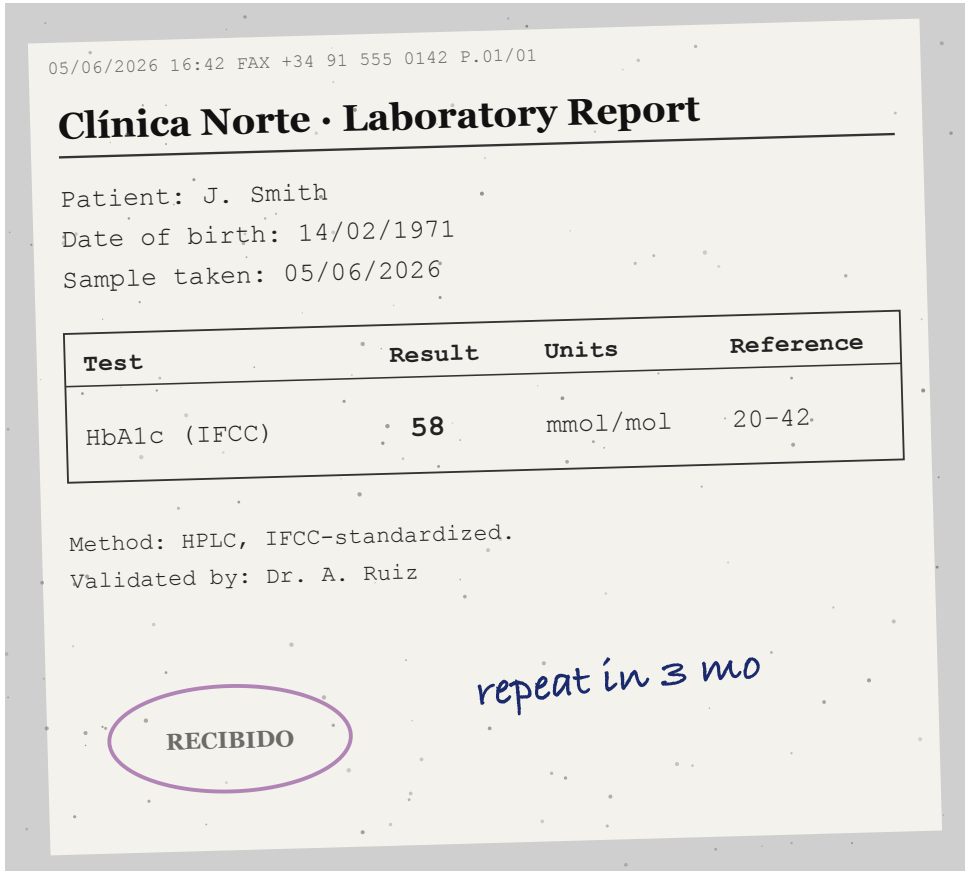


A: x 100–140, y 100–140
B: x 110–150, y 110–150

area A = area B = 1,600
A ∩ B = 30 × 30 = 900
A ∪ B = 1,600 + 1,600 − 900
= 2,300

IoU = 900 / 2,300 ≈ 0.39

Form Images



OCR output: one row per word

text	x	y	conf
HbA1c	52	278	96
(IFCC)	112	278	91
58	262	278	88
mmol/mol	350	278	93

Optical character recognition (OCR) gives words, boxes, and confidence. Layout links “58” to “HbA1c”; the Week 5 pipeline then normalizes it to 7.5%.

Extracting a Form Image with a Vision Model

```
b64 = base64.b64encode(open("report-b.png", "rb").read()).decode()
r = client.chat.completions.create(
    model="gpt-5.6-luna",
    response_format={"type": "json_object"},
    messages=[{"role": "user", "content": [
        {"type": "text", "text": "Return JSON with test, value, unit, sample_date, exactly as written."},
        {"type": "image_url", "image_url": {"url": f"{b64}"}}]}])
fields = json.loads(r.choices[0].message.content)
```

```
{"test": "HbA1c (IFCC)", "value": 58, "unit": "mmol/mol", "sample_date": "05/06/2026"}
```

- The date is still ambiguous: Week 5's normalize step applies unchanged
- A misread digit (58 as 38) is fluent and plausible; range checks send rows to the review queue
- Store the image hash, model, and prompt with each extracted row

Failure Modes in Image Data

- **Shortcut learning:** pneumonia models on chest X-rays could identify the source hospital (over 95% accuracy); AUC fell from 0.93 to 0.82 at another hospital
- **Confounding marks:** skin markings cut a melanoma classifier's specificity from 84.1% to 45.8%
- **Unrepresented groups:** three dermatology models had AUC 0.56–0.67 on a diverse image set, lowest on the darkest skin tones
- **Metadata errors:** a wrong laterality tag or a missing EXIF rotation compares the wrong eye or a sideways photo
- **Text in pixels:** names burned into an image survive removal of identifiers from the header

Zech et al., PLOS Medicine (2018); Winkler et al., JAMA Dermatology (2019); Daneshjou et al., Science Advances (2022)

Summary

Time series

- Order is part of the data; time zones and gaps change answers
- Windows (tumbling, sliding), `LAG`, and `ASOF` joins express most questions in SQL
- Events are runs of rows; similar windows are a vector search

Images

- Pixels plus metadata; query the metadata and derived rows, not the pixels
- Embeddings search images by text or by example
- VLMs and detectors turn pixels into text and rows that SQL can query

Thanks for the Attention
Q&A